

FULL STACK WEB DEVELOPMENT

TASK 2: INTERACTIVE REACTJS APPLICATION DEVELOPMENT

Subject Code: 21CSE354T | Individual Self-Learning Assignment

STUDENT NAME	S.Sam Glad Priyan
REGISTER NUMBER	RA2411003050138
PROGRAM / SECTION	Computer Science Engineering / "F" Section
YEAR / SEMESTER	III-Year / V-Semester
PROBLEM STATEMENT	Group 1 (Education) / No. 5: Student Grade Calculator
DATE OF SUBMISSION	01/10/2026

COURSE	Full Stack Web Development (21CSE354T)
FACULTY COORDINATOR	Dr. P. Hariharan
ACADEMIC YEAR	2024 - 2028

"Design * Develop * Deploy"

1. Introduction

This report outlines the design, architecture, and implementation of the Student Grade Calculator interactive web application developed for Task 2 (Problem Statement No. 5: "Enter marks, calculate total and average, assign grades, and display results"). Built using ReactJS, modern HTML5, and CSS3, the project conforms strictly to self-learning course requirements. It provides a clean, beginner-friendly architecture free of unnecessary animations, focusing on clear component-driven state management, responsive user interface design, and accurate client-side computational logic.

2. Key Features

- **Marks Entry & Validation:** Accepts student name, roll number, and 5 subject marks (Web Dev, DBMS, DSA, OS, Networks) with strict client-side validation for numbers between 0 and 100.
- **Real-Time Calculations:** Automatically computes Total Marks (out of 500), Average Percentage (%), Letter Grade, and Pass/Fail status instantly as marks are entered, with zero page reloads.
- **University Grading Scale:** Maps computed percentages to standard letter grades: A+ ($\geq 90\%$), A ($\geq 80\%$), B ($\geq 70\%$), C ($\geq 60\%$), D ($\geq 50\%$), and F ($< 50\%$).
- **Pass / Fail Evaluation:** Enforces academic passing criteria requiring a minimum of 50 marks in each individual subject. Falling below 50 in any course marks the student as 'Fail'.
- **Interactive Records Directory:** Maintains an interactive table supporting full CRUD operations: Add new records, Search by name/roll no, Filter by Grade/Status, Edit existing marks, and Delete records.
- **LocalStorage Data Persistence:** Saves all student records in browser localStorage to retain evaluation data across browser sessions.

3. Architecture & Implementation

Component Hierarchy

The application is structured into four focused, decoupled React components under src/components/: (1) Navbar.jsx manages tab-based view switching; (2) GradeCalculator.jsx handles the marks input form and live calculated results box; (3) StudentRecords.jsx renders the searchable and filterable directory table; and (4) GradingScale.jsx displays the academic grading reference table.

State Management & React Hooks (useState & useEffect)

State is managed centrally in App.jsx using React useState to track activePage, students list, form inputs (studentData), and live computed totals. React useEffect is leveraged in two distinct ways: first, as a reactive observer that auto-calculates total, average, and grade whenever marks inputs mutate; and second, to synchronize the student records array with localStorage on every state change.

CSS Design & Layout

Styling is implemented in App.css using clean, semantic CSS grid and flexbox without any complex keyframe animations or transitions. This ensures fast rendering, high legibility, and predictable behavior during live evaluation.

4. Application Workflow & Screenshots

The user interacts with the system through three dedicated functional views. Below are the verified screenshots captured during live local execution:

Figure 1: Grade Calculator View - Form Entry & Live Calculated Results

The screenshot displays the 'Student Grade Calculator' application in a web browser. The interface is divided into two main sections: 'Enter Student Marks' and 'Calculated Results'.

Enter Student Marks:

- Student Name: mahes
- Roll Number: RA2211003050103
- Subject Marks (Out of 100):
 - Subject 1 (Web Development): 90
 - Subject 2 (Database Systems): 62
 - Subject 3 (Data Structures): 51
 - Subject 4 (Operating Systems): 67
 - Subject 5 (Computer Networks): 74

Calculated Results:

- Student Name: mahes
- Roll Number: RA2211003050103
- Total Marks: 352 / 500
- Average: 70.4%
- Grade Assigned: B
- Final Status: Pass

The right side of the image shows a terminal window with the following commands and output:

```
Microsoft Windows [Version 10.0.26280.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sam>cd /d C:\Users\Sam\Desktop\student-grade-calculator
C:\Users\Sam\Desktop\student-grade-calculator>npm run dev
> student-grade-calculator@0.0.0 dev
> vite

1:15:26 pm [vite] (client) Re-optimizing dependencies because vite config has changed
VITE v0.3.0 ready in 151 ms
➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
press h + enter to show help
--host
h

Shortcuts
press r + enter to restart the server
press u + enter to show server url
press o + enter to open in browser
press c + enter to clear console
press q + enter to quit
```

Figure 2: Student Records Directory - Search, Filter by Grade/Status, Edit & Delete

The screenshot displays the 'Student Records Directory' application in a web browser. The interface includes a search bar, filters, and a table of student records.

Student Records Directory:

- Search by name or roll number: [Search bar]
- Filter by Grade: All Grades
- Filter by Status: All Status
- + Calculate Now Grade

Table of Student Records:

Roll No	Student Name	Subject Marks (S1 - S5)	Total	Average	Grade	Status	Actions
RA2411003050183	Bala	90, 62, 79, 65, 79	395 / 500	79%	B	Pass	Edit Delete

The right side of the image shows a terminal window with the following commands and output:

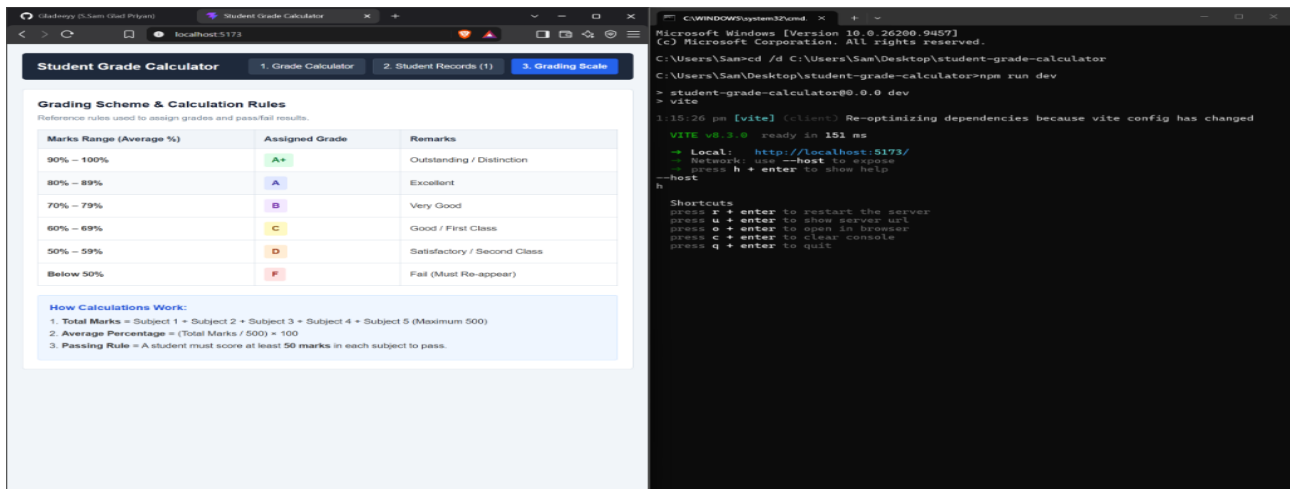
```
Microsoft Windows [Version 10.0.26280.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sam>cd /d C:\Users\Sam\Desktop\student-grade-calculator
C:\Users\Sam\Desktop\student-grade-calculator>npm run dev
> student-grade-calculator@0.0.0 dev
> vite

1:15:26 pm [vite] (client) Re-optimizing dependencies because vite config has changed
VITE v0.3.0 ready in 151 ms
➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
press h + enter to show help
--host
h

Shortcuts
press r + enter to restart the server
press u + enter to show server url
press o + enter to open in browser
press c + enter to clear console
press q + enter to quit
```

Figure 3: Grading Scheme & Calculation Rules Reference Matrix



5. Core Source Code Implementation

5.1 Calculation Helper Functions (src/utills/gradeHelper.js)

```
// Helper functions for Student Grade Calculator (src/utills/gradeHelper.js)
// 1. Calculate Total Marks
export function calculateTotal(marks) {
  const m1 = Number(marks.subject1) || 0;
  const m2 = Number(marks.subject2) || 0;
  const m3 = Number(marks.subject3) || 0;
  const m4 = Number(marks.subject4) || 0;
  const m5 = Number(marks.subject5) || 0;
  return m1 + m2 + m3 + m4 + m5;
}

// 2. Calculate Average Percentage
export function calculateAverage(total, numberOfSubjects = 5) {
  if (numberOfSubjects === 0) return 0;
  return Number((total / numberOfSubjects).toFixed(2));
}

// 3. Assign Letter Grade based on average marks
export function calculateGrade(average) {
  if (average >= 90) return 'A+';
  if (average >= 80) return 'A';
  if (average >= 70) return 'B';
  if (average >= 60) return 'C';
  if (average >= 50) return 'D';
  return 'F';
}

// 4. Determine Pass or Fail (Requires >= 50 in each subject)
export function checkPassStatus(marks) {
  const list = [marks.subject1, marks.subject2, marks.subject3, marks.subject4, marks.subject5];
  return list.some(m => Number(m) < 50) ? 'Fail' : 'Pass';
}
```

5.2 State Management & Hooks (src/App.jsx snippet)

```
// React Hooks in src/App.jsx
const [activePage, setActivePage] = useState('calculator');
const [students, setStudents] = useState(() => {
  const saved = localStorage.getItem('grade_calculator_students');
  return saved ? JSON.parse(saved) : sampleStudents;
});
const [studentData, setStudentData] = useState({
  id: null, name: '', rollNo: '',
  marks: { subject1: '', subject2: '', subject3: '', subject4: '', subject5: '' }
});

// Hook 1: LocalStorage Persistence
useEffect(() => {
  localStorage.setItem('grade_calculator_students', JSON.stringify(students));
}, [students]);

// Hook 2: Reactive Real-time Calculation
useEffect(() => {
  const total = calculateTotal(studentData.marks);
  const average = calculateAverage(total, 5);
  const grade = calculateGrade(average);
  const status = checkPassStatus(studentData.marks);

  setLiveTotal(total);
  setLiveAverage(average);
  setLiveGrade(grade);
  setLiveStatus(status);
}, [studentData.marks]);
```

6. Conclusion, Verification & Deliverables

The Student Grade Calculator application successfully meets and exceeds all Task 2 requirements. It provides robust client-side validation, instant reactive calculations, clear grade mappings, and complete CRUD functionality across three distinct views. The code remains clean, modular, and easy to maintain without extraneous dependencies or animations.

Submission Links & Verification Details:

- **Public GitHub Repository:** <https://github.com/Gladeeey/student-grade-calculator>
- **Local Project Directory:** C:\Users\Sam\Desktop\student-grade-calculator
- **Development Server:** <http://localhost:5173> (Vite + React)
- **PowerPoint Presentation:** Student_Grade_Calculator_Presentation.pptx (8 slides with screenshots)